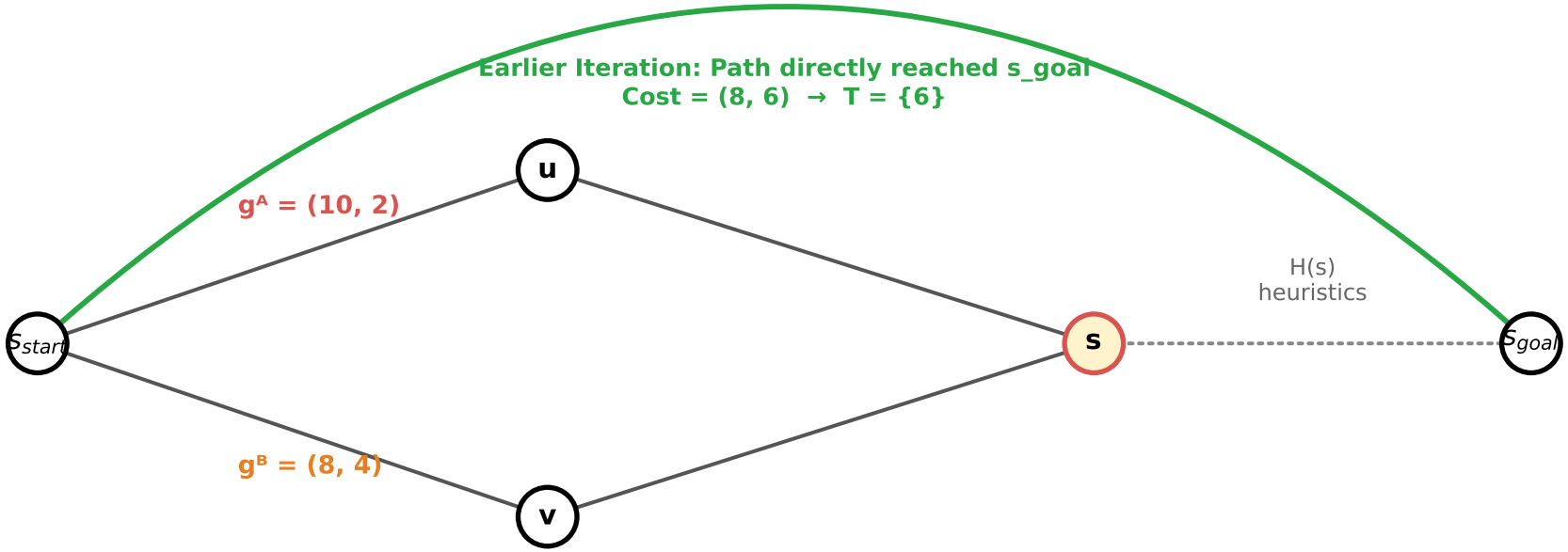
Maya’s Fallback Algorithm

Why We Truncate Dimensions — And Why We Still Need a Full Check

A Complete, Step-by-Step Algorithmic Trace (from Ground Zero)

Step 0: The Graph & The Origin of T = {6}

Where did our previous paths and the goal frontier come from?



0: Previous search history before arriving at node s:

1. An earlier path already reached s_goal with total cost (8, 6). The truncated cost enters T = {6}.
2. Two new paths are now approaching state s: Path A with g^A = (10, 2) and Path B with g^B = (8, 4).

What Does Goal Frontier T = {6} Mean?

How known solutions prune future paths via CHOOSEH

1. What is stored in T?

- T is the truncated closed set at the destination: $T = G_cl_Tr(s_goal)$.
- Truncation omits the primary coordinate g_1 and keeps the rest: $Tr(g) = g_2$.
- Since a complete path already reached s_goal with cost $g_2 = 6$, we have: $T = \{6\}$.
- Fundamental Invariant: Any solution already at s_goal represents an achievable total cost.

2. How does CHOOSEH prune using T?

- For any partial path arriving at state s with cost g, testing heuristic h gives: $f = g + h$.
- The projected cost in dimension 2 is: $Tr(f) = g_2 + h_2$.
- If $t \leq Tr(f)$ for any $t \in T$ (here: $6 \leq g_2 + h_2$):

  → Completing this path via h will cost AT LEAST 6 in dimension 2!

  → But we already discovered a complete path to the goal with cost 6!

  → Therefore, this heuristic completion CANNOT produce an undominated solution.

  → Heuristic h is safely REJECTED.

Goal Pruning Rule:

If $6 \leq g_2 + h_2$, completing through heuristic h is strictly worse than an existing solution.

The Core Concept: Truncation & The Fallback

Why compare M-1 dimensions at all? And why does it break?

1. Why use Truncation at all?

- Normal Pareto dominance checks all M coordinates: $g'_1 \leq g_1$ AND $g'_2 \leq g_2$.
- In single-heuristic A*, OPEN pops nodes in strictly increasing order of primary cost g_1.
- If g_1 always arrives in non-decreasing order (monotonic), $g'_1 \leq g_1$ is ALWAYS TRUE for past nodes!
- We can drop dimension 1 and only compare $Tr(g) = g_2$ → massive speedup across millions of queries.

2. Why do we still need a Full-Dimensional Check (Fallback)?

- Multi-Valued Heuristics (MVH) assign multiple lower bounds: $H(s) = \{h^{(1)}, h^{(2)}\}$.
- Different paths choose different heuristics → their $f_1 = g_1 + h_1$ priority key shifts in OPEN!
- A path with worse actual cost ($g_1 = 10$) can pop BEFORE a path with better cost ($g_1 = 8$).
- When the second path arrives, Truncation sees only g_2 and claims Path A dominates it (FALSE DOMINANCE)!
- The Fallback catches this violation ($\max g'_1 \not\leq g_1$) and re-checks all M dimensions.

Key Insight:

Truncation is an optimistic speedup. The Fallback guarantees exact Pareto soundness.

Maya’s LOCALDOMCHECK Algorithm

(Wolff, Felner, Salzman, SoCS 2026)

```
function LOCALDOMCHECK(s, g):
    // 1. Optimistic Truncated Check (in M-1 dimensions):
    if exists g' in G_cl_Tr(s) such that Tr(g') <= Tr(g):

        // 2. Monotonicity Guard (is g₁ order valid?):
        if max_g'1 <= g1:
            return TRUE    // Soundly dominated via Truncation

        // 3. FALLBACK: Full-Dimensional Check (all M dimensions):
        if exists g' in G_cl(s) such that g' <= g:
            return TRUE    // Truly dominated in full space
        return FALSE       // Non-dominated! Saved by fallback!

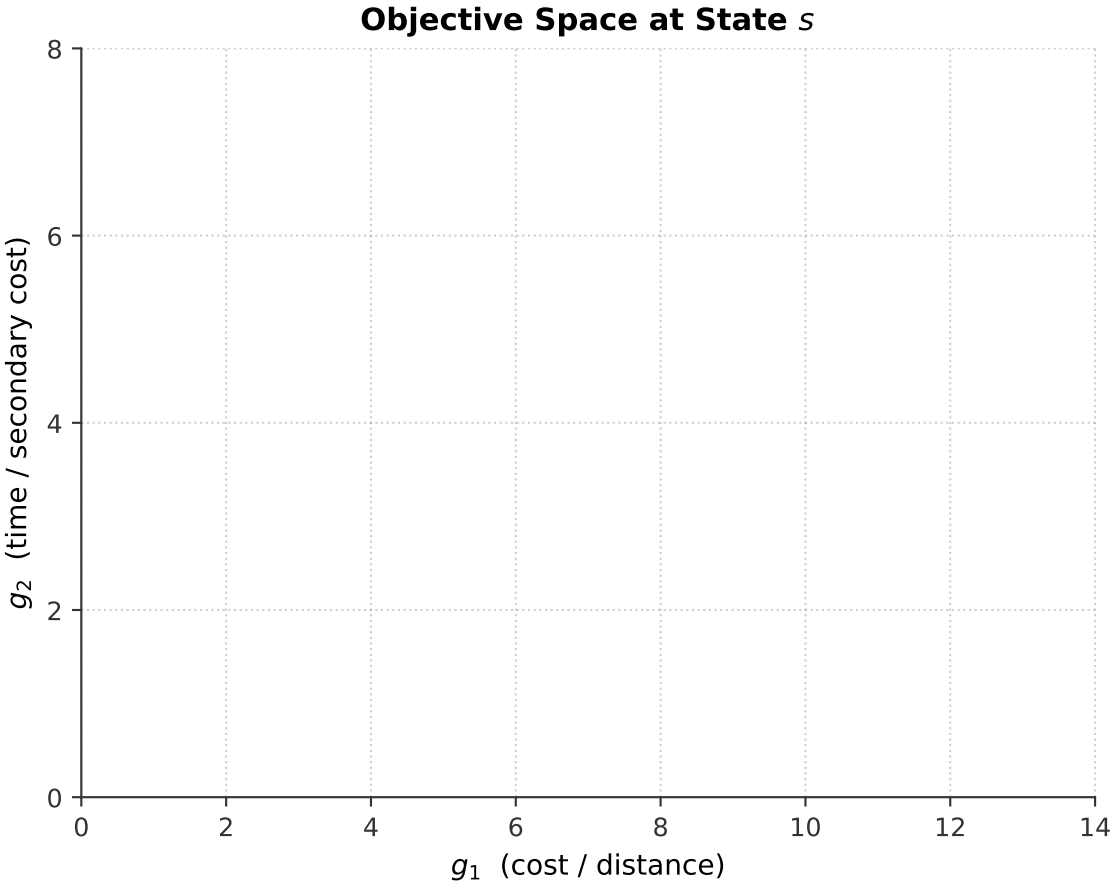
    return FALSE           // Undominated
```

The 3 Steps:

1. Fast test (M-1 dims) → 2. Guard (max $g'_1 \le g_1$) → 3. Fallback (all M dims).

Search State at Node s

Before either path expands at node s



SEARCH STATE & VARIABLES

Heuristics at state s (sorted ascending by h_1):

- $h^{(1)} = (1, 3)$ → optimistic in g_1 (1), moderate in g_2 (3)
- $h^{(2)} = (5, 1)$ → pessimistic in g_1 (5), optimistic in g_2 (1)

Incoming Paths to state s:

- Path A: $g^A = (10, 2)$
- Path B: $g^B = (8, 4)$

Notice: Neither dominates the other! (10 > 8, but 2 < 4).

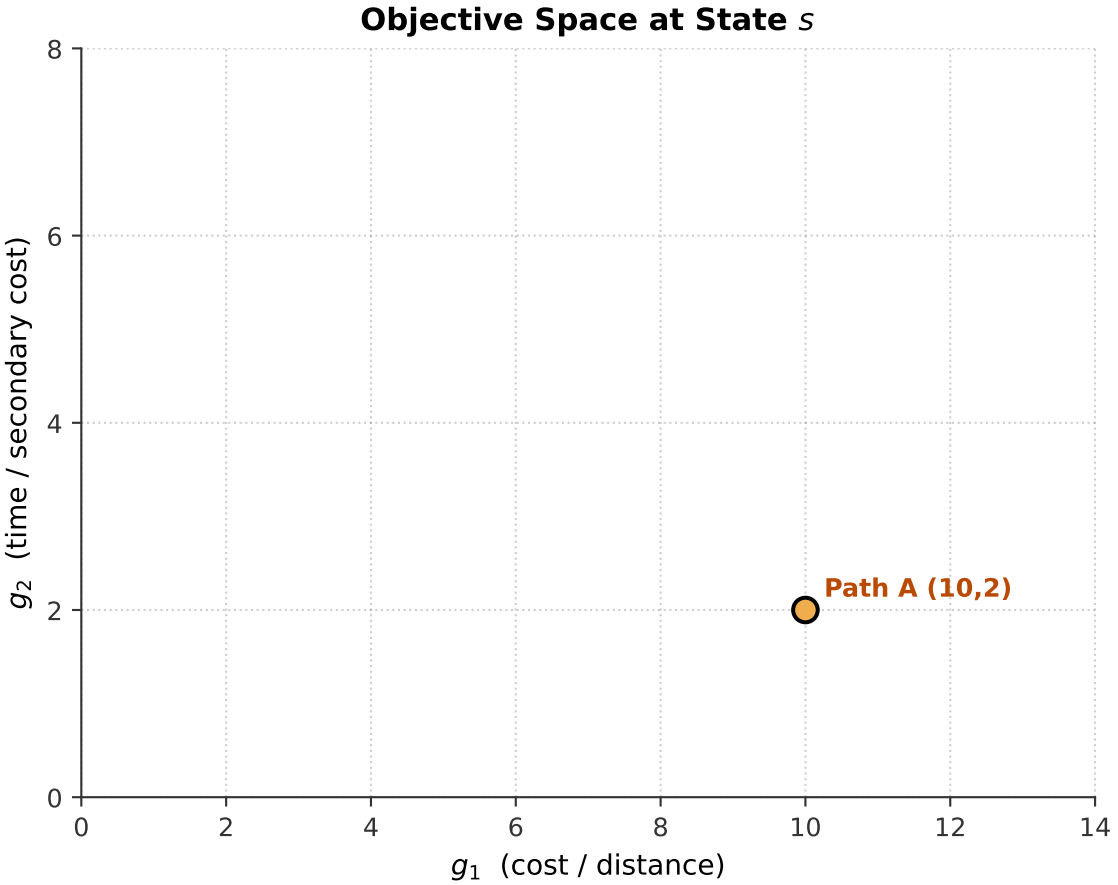
G_cl(s): ∅

0: Initial state at node s. G_cl(s) is empty, Goal frontier T = {6}.

Note: Both Path A and Path B are approaching state s.

Step 1: Path A Arrives at State s

$g^A = (10, 2)$ evaluates heuristics via CHOOSEH



SEARCH STATE & VARIABLES

CHOOSEH for Path A:
1. Test $h^{(1)} = (1, 3)$:
   $f = g^A + h^{(1)} = (10+1, 2+3) = (11, 5)$
   $Tr(f) = f_2 = 5$
2. Check vs Goal $T = \{6\}$:
   Is $6 \leq 5$? → FALSE (Not dominated by goal)
3. $h^{(1)}$ is Accepted!

Path A enters OPEN with priority key: $f_1 = 11$.

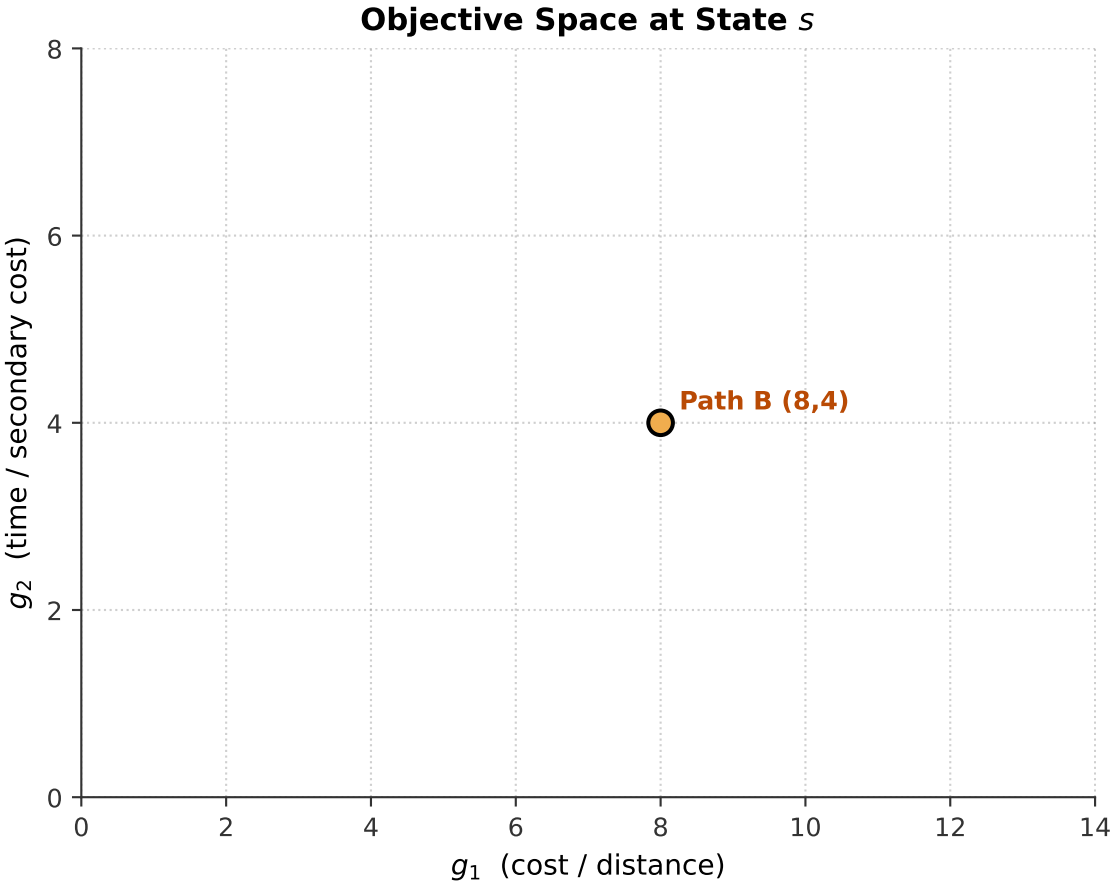
G_cl(s): ⌀

1: Path A selects $h^{(1)}$ and enters OPEN with priority $f_1 = 11$.

Note: Path A has actual accumulated cost $g_1 = 10$.

Step 2a: Path B Arrives at State s

$g^B = (8, 4)$ — Why is $h^{(1)}$ refuted? Because of the GOAL, not Path A!



SEARCH STATE & VARIABLES

CHOOSEH for Path B:

1. Test $h^{(1)} = (1, 3)$:
   $f = g^B + h^{(1)} = (8+1, 4+3) = (9, 7)$
   $Tr(f) = f_2 = 7$
2. Check vs Goal $T = \{6\}$:
   Is $6 \leq 7$? → TRUE (Dominated by Goal!)

★ WHY is it rejected even though $g_1^B = 8 < 10$?

- It is NOT being compared to Path A!
- It is compared to the GOAL solution in dimension 2!
- Completing Path B via $h^{(1)}$ takes time 7, but the goal was already reached with time 6!

→ $h^{(1)}$ is discarded for Path B.

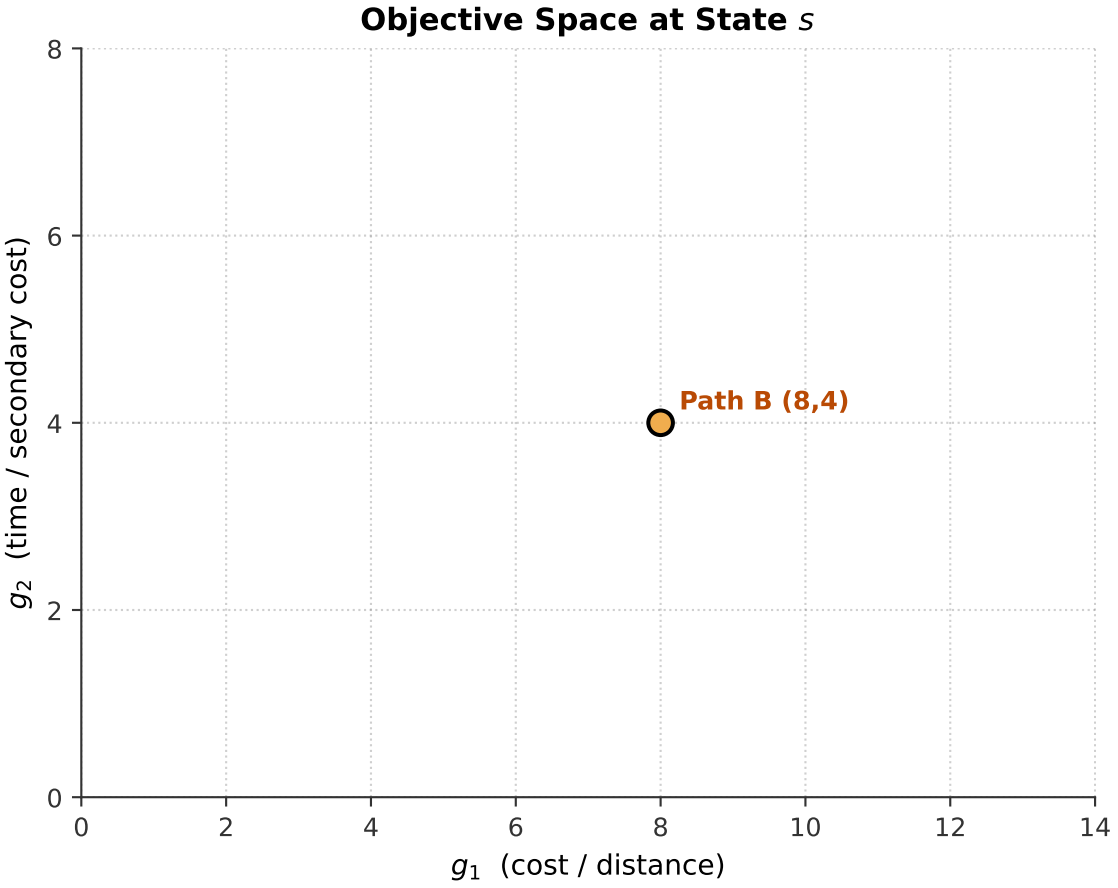
G_cl(s): ∅

2a: Path B cannot use $h^{(1)}$ because projected time 7 cannot beat known goal time 6 ($6 \leq 7$).

Note: Path B is forced to test its second heuristic $h^{(2)}$.

Step 2b: Path B Accepts $h^{(2)}$ & Enqueues

$g^B = (8, 4)$ paired with $h^{(2)} = (5, 1) \rightarrow$ priority $f_1 = 8 + 5 = 13$



SEARCH STATE & VARIABLES

CHOOSEH for Path B (cont.):
1. Test $h^{(2)} = (5, 1)$:
   $f = g^B + h^{(2)} = (8+5, 4+1) = (13, 5)$
   $Tr(f) = f_2 = 5$
2. Check vs Goal $T = \{6\}$:
   Is $6 \leq 5$? → FALSE (Not dominated)
3. $h^{(2)}$ Accepted! Enqueued with $f_1 = 13$.

★ PRIORITY INVERSION IN OPEN:
Path A has lower f_1 ($11 < 13$) → POPPED FIRST!
Even though Path B has BETTER actual cost ($8 < 10$)!

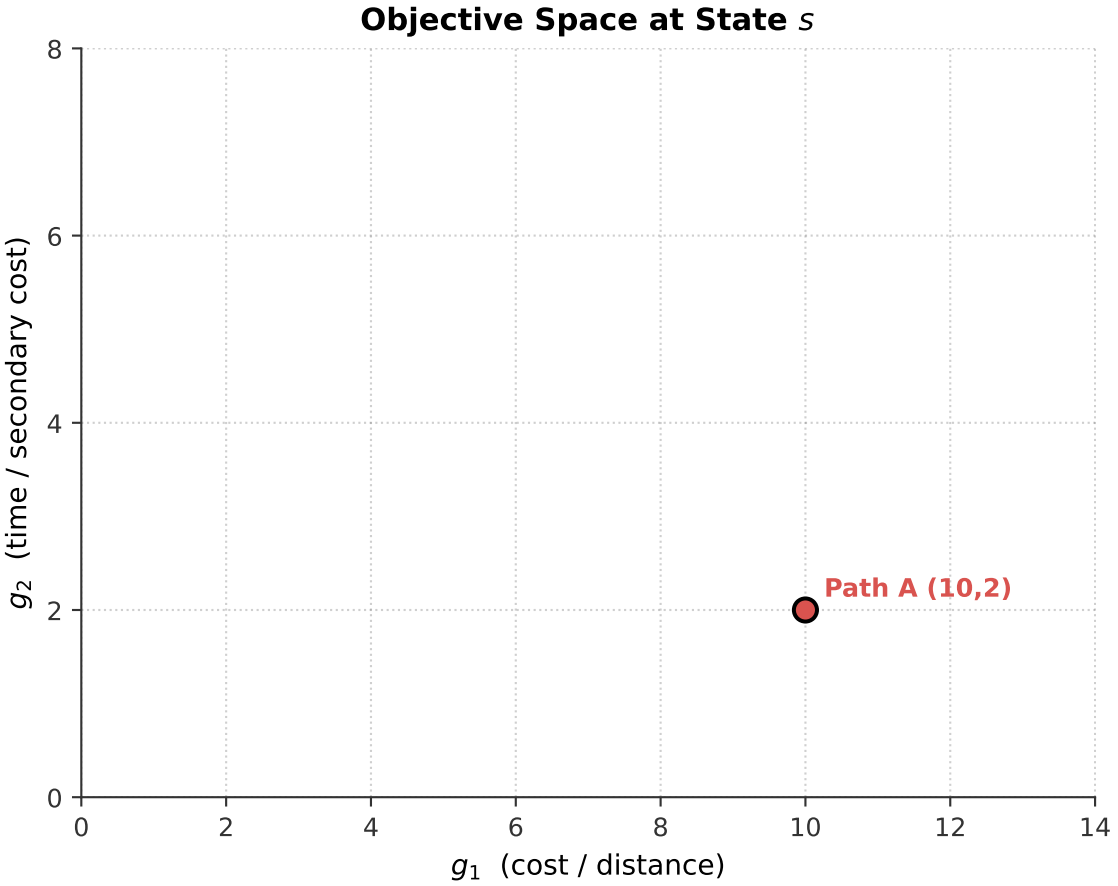
G_cl(s): ∅

2b: Path B is enqueued behind Path A in OPEN because $f_1^B = 13 > f_1^A = 11$.

Note: ORDER INVERSION: Lower g_1 path will arrive at state s SECOND!

Step 3: Path A Popped & Expanded

OPEN pops min f_1: Path A ($f_1 = 11$) commits to state s



SEARCH STATE & VARIABLES

Execution Trace:
- OPEN.PopMin() returns Path A ($f_1 = 11$)
- LOCALDOMCHECK: G_cl is empty → FALSE
- UPDATE(s, (10, 2)):
  - G_cl(s) ← {(10, 2)}
  - G_cl_Tr(s) ← {2}
  - max g'_1 ← 10 (recorded in state)
- Path A expanded; successors generated.

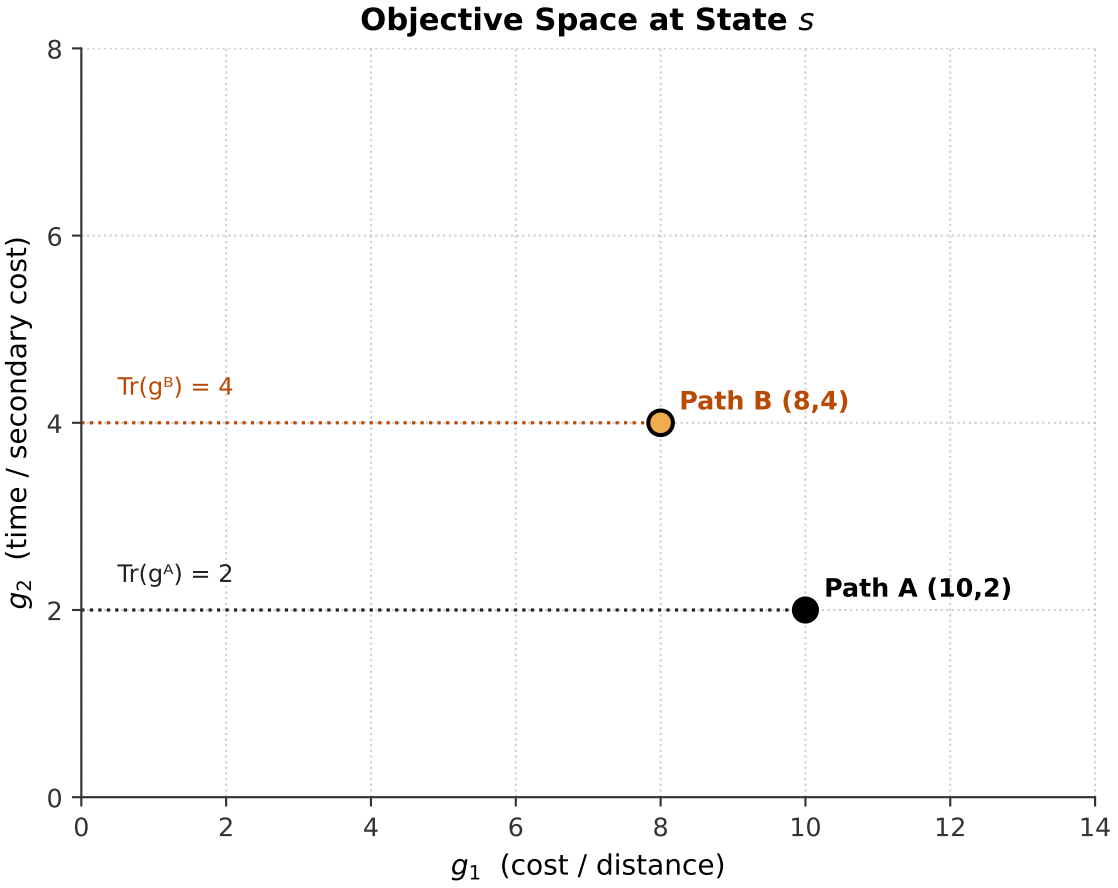
G_cl(s): `{(10, 2)}`

3: Path A is closed at state s. G_cl(s) = {(10, 2)} and max g'_1 = 10.

Note: Next node to pop from OPEN is Path B with $g^B = (8, 4)$.

Step 4a: Path B Popped — Truncated Check

Query: Does any g' in G_cl_Tr(s) have Tr(g') ≤ Tr(g^B)?



SEARCH STATE & VARIABLES

Step 1 of LOCALDOMCHECK (Truncated Check):
- $Tr(g^B) = g_2^B = 4$
- Compare against G_cl_Tr(s) = {2}:
  Is 2 ≤ 4 ?
  → TRUE!

⚠ DANGER:
Looking ONLY at coordinate g_2, Path A appears to dominate Path B
If we prune now, Path B is erroneously discarded.

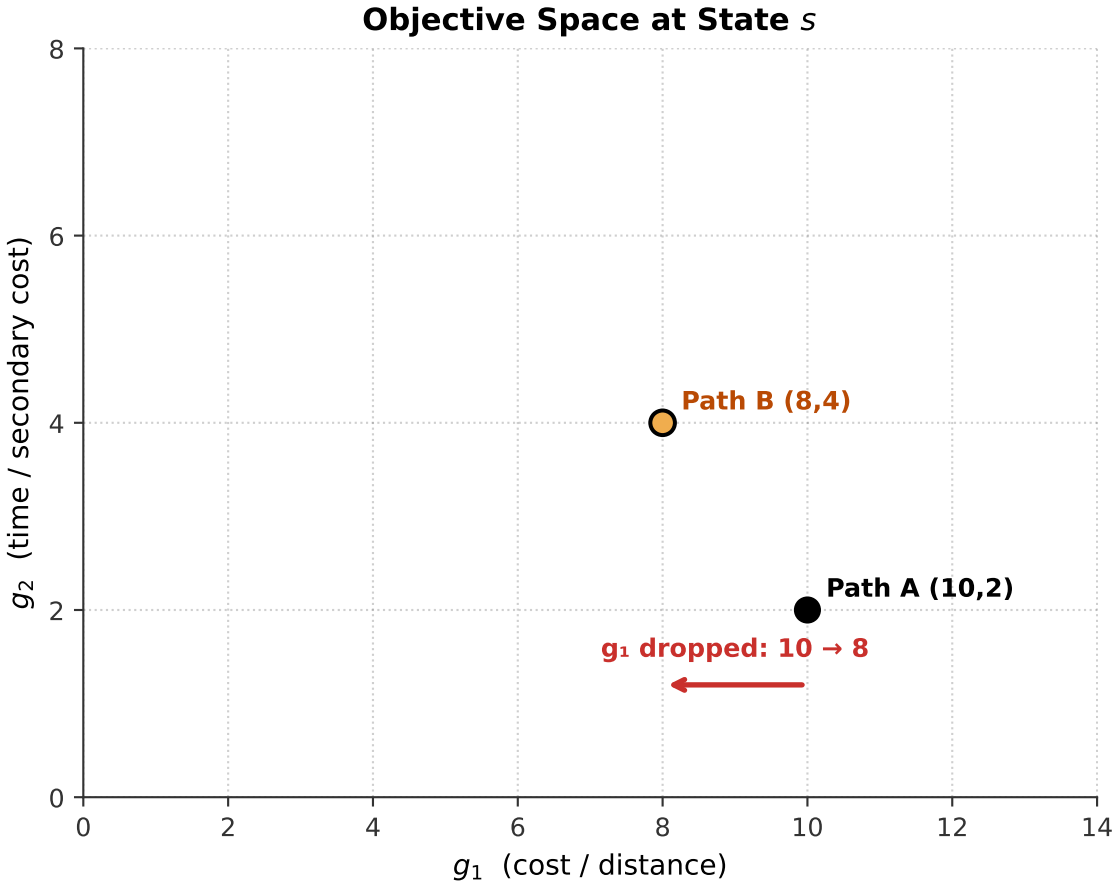
G_cl(s): {(10, 2)}

4a: Truncated test succeeds (2 ≤ 4). Path B appears dominated in g_2.

Note: Truncation assumes g_1 was monotonic. Is it?

Step 4b: Monotonicity Guard Catches the Trap

Condition: Does max $g'_1 \leq g_1^B$ hold?



SEARCH STATE & VARIABLES

Step 2 of LOCALDOMCHECK (Monotonicity Guard):
- Recorded max $g'_1 = 10$ (from Path A)
- Incoming Path B has $g_1^B = 8$
- Check: Is max $g'_1 \leq g_1^B$?
  Is $10 \leq 8$?
  → FALSE! (Order Violation!)

✓ The algorithm detects the truncated test was misled!
→ FALLBACK TO FULL DIMENSIONS TRIGGERED.

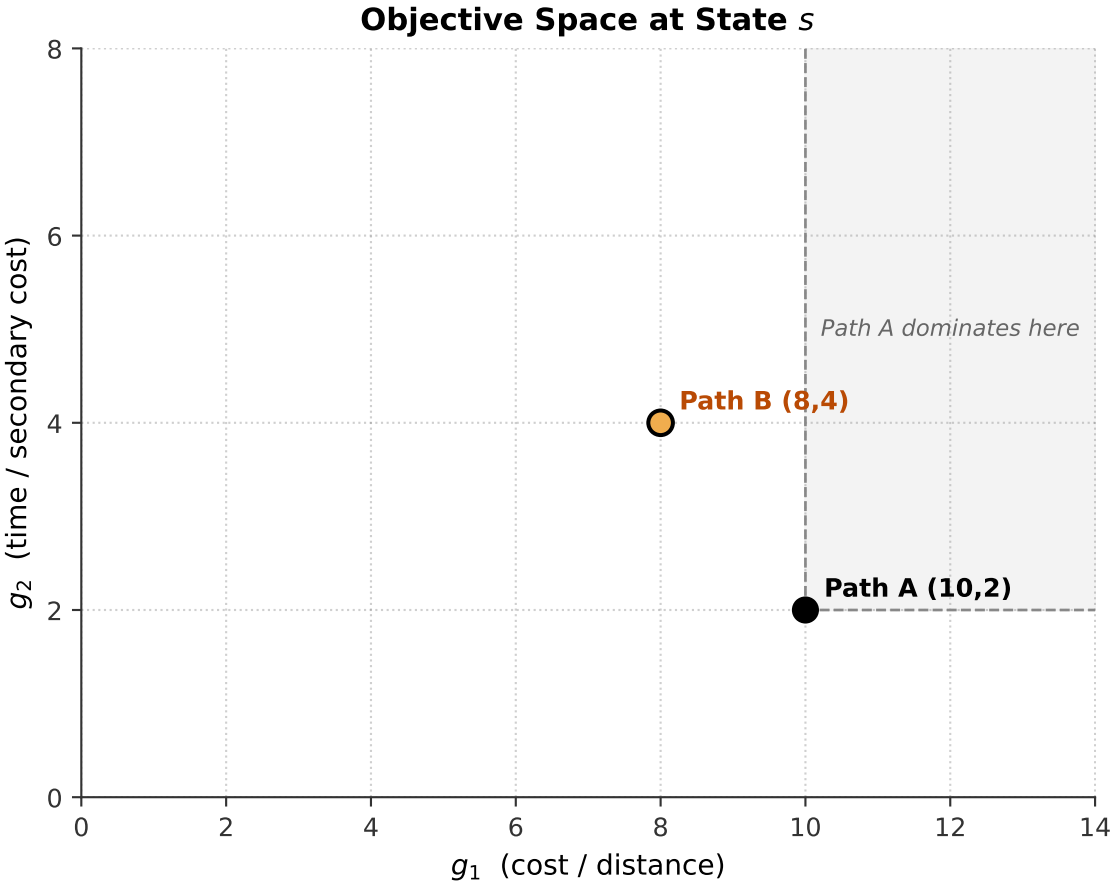
G_cl(s): `{(10, 2)}`

4b: Monotonicity guard detects order violation ($10 \not\leq 8$).

Note: Dimensionality reduction is inconclusive. We cannot safely prune.

Step 4c: Fallback — Full M-Dimensional Check

Compare Path A (10, 2) vs Path B (8, 4) in all M = 2 coordinates



SEARCH STATE & VARIABLES

Step 3 of LOCALDOMCHECK (Full Fallback):
Compare (10, 2) vs (8, 4) in full space:
• Dim 1 (g_1): Is 10 ≤ 8? → FALSE (Path B is better!)
• Dim 2 (g_2): Is 2 ≤ 4? → TRUE (Path A is better)

Decision:
• Path A does NOT dominate Path B!
• This is a genuine Pareto trade-off.
• LOCALDOMCHECK returns FALSE → Path B SURVIVES!

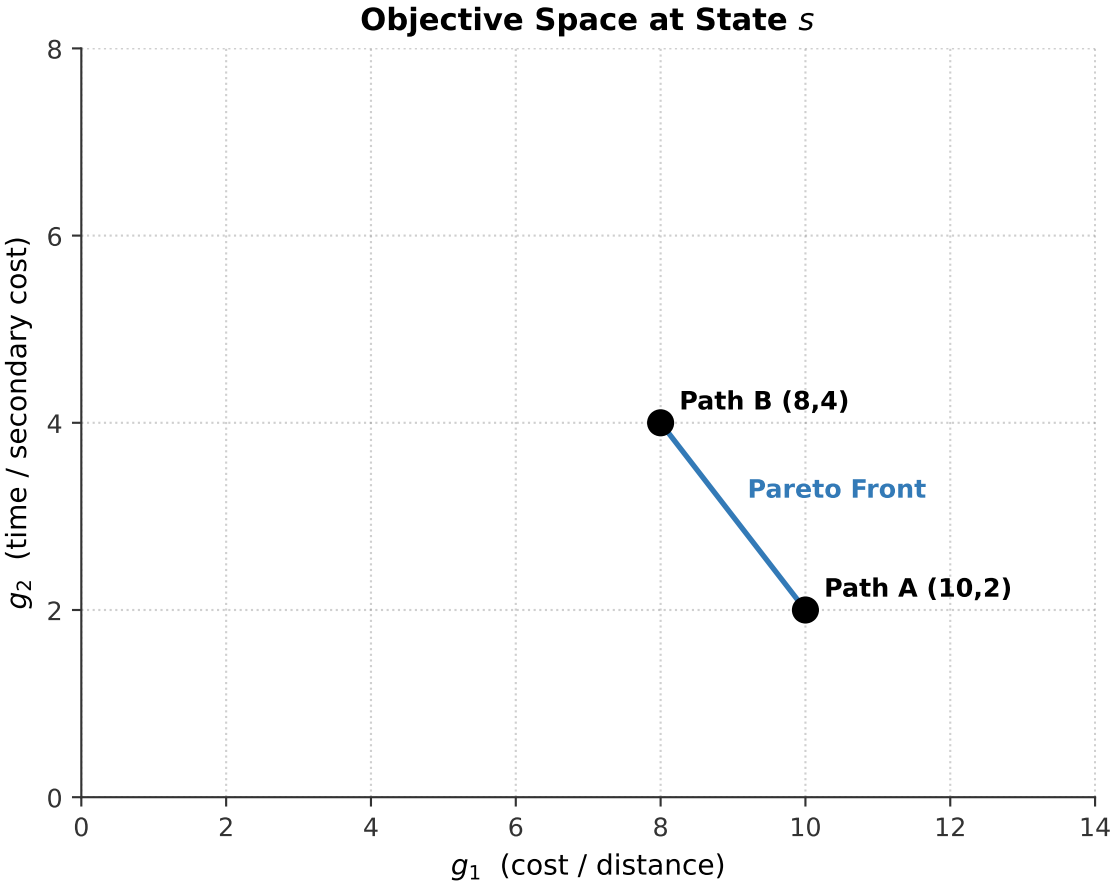
G_cl(s): `{(10, 2)}`

4c: Full check confirms Path B is NOT dominated. Path B is saved!

Note: Notice Path B lies outside Path A's dominance cone (shaded gray).

Step 4d: Both Paths on Pareto Front

G_cl(s) = {(10, 2), (8, 4)} — search soundness preserved



SEARCH STATE & VARIABLES

Search State Consolidated:

- UPDATE commits Path B:
  - G_cl(s) = {(10, 2), (8, 4)}
  - G_cl_Tr(s) = {2, 4}
- Both paths continue expanding toward the goal.
- Exact Pareto optimality is preserved bit-identically!

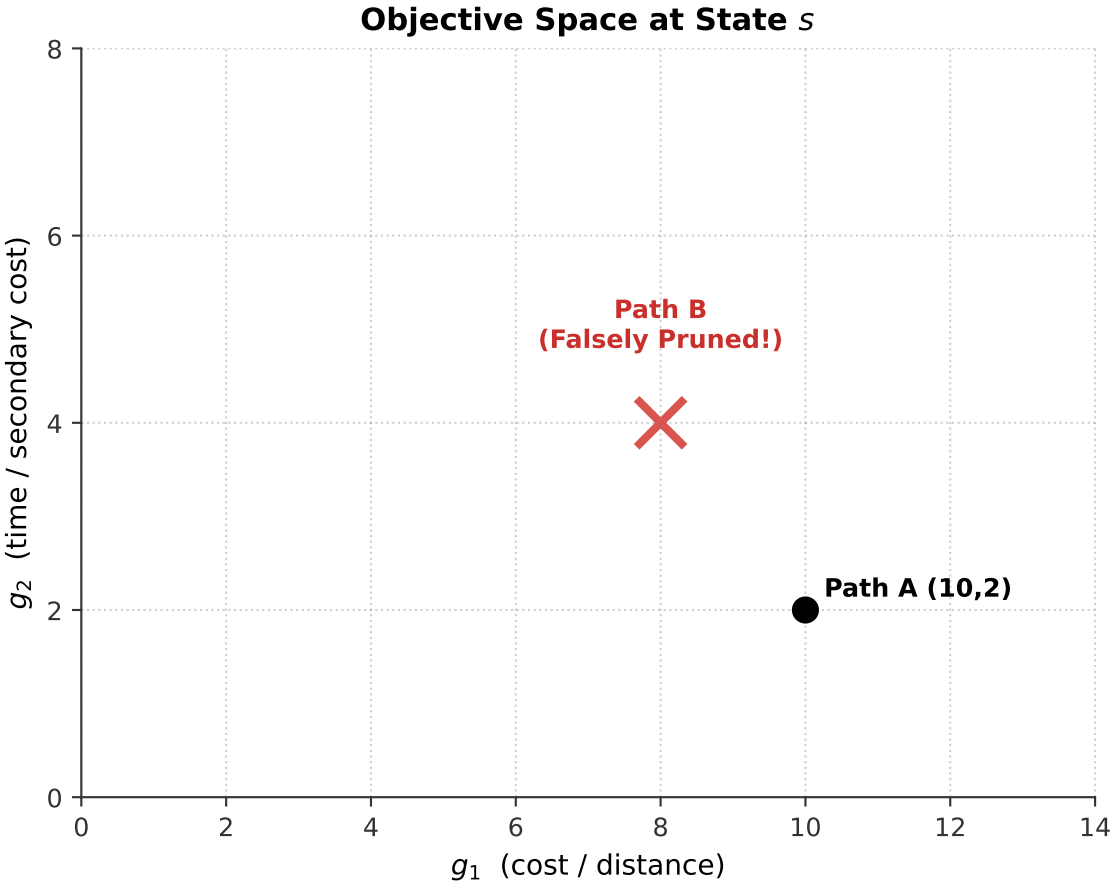
G_cl(s): `{(10, 2), (8, 4)}`

4d: Both Path A and Path B survive and enter the closed set.

Note: Search soundness is preserved 100% bit-identically.

Failure Case: What If We Omit the Fallback?

Naive Truncation murders valid Pareto solutions



SEARCH STATE & VARIABLES

Without Fallback:
1. Truncated test sees 2 ≤ 4 (True).
2. Algorithm prunes Path B immediately.
3. Path B never reaches the destination.

Consequence:
- Pareto front returned is INCOMPLETE.
- Optimal solutions in g_1 are permanently lost.
- Search algorithm is MATHEMATICALLY UNSOUND.

G_cl(s): `{(10, 2)}`

Fatal Error: Pruning on truncation alone discards optimal solutions.

Note: Maya's fallback is not an optimization — it is a strict correctness requirement.

Accelerating the Fallback: Full-Dim K-d Tree

From O(N) linear scan to O(N^(1 - 1/M)) spatial pruning

The Bottleneck in Baseline (FAST2):

- The fallback re-checks all M dimensions against every vector in G_cl(s).
- In 98.5% of fallbacks, the candidate is non-dominated → must scan all N vectors!
- On 6-objective benchmarks: 1.35 BILLION comparisons, taking 85%+ of search time.

The FAST-MVH Solution (Full-Dimensional K-d Tree):

- Index G_cl(s) in a K-d Tree over all M dimensions.
- Each node stores subtree minimum bounds: lo = (min p_1, ..., min p_M).
- O(1) Bounding-Box Pruning: If lo[k] > g[k] for ANY coordinate k,
  no vector in that subtree can dominate g → skip the entire branch!
- Result: 217× fewer comparisons, 9× wall-clock speedup, 100% bit-identical front.

FAST-MVH: Replaces the linear fallback scan with a K-d tree for massive speedups.

Note: Same nodes expanded, same solutions returned — pure algorithmic acceleration.

Summary: Why Truncate & Why Fallback?

The core intuition in three key points

1. Why Truncate?	OPEN sorts by $f_1 = g_1 + h_1$. Checking only M-1 dimensions saves massive work whenever g_1 arrives in non-decreasing order.
2. Why Fallback?	MVH evaluates paths with different heuristics, causing paths to arrive out of g_1 order. Truncation falsely flags them as dominated; the Fallback re-checks all M dimensions to preserve Pareto optimality.
3. Why K-d Tree?	Linear fallback scans consume 85%+ of runtime. Spatial K-d trees prune subtrees in O(1), making the fallback fast and scalable.

End of Presentation.

Questions?